```
      email.attachments.each do |attachment|
        page.attachments.create({
          file: attachment,
          description: email.subject
        })
      end
    end
  end
end
```

4 Action Mailer Callbacks

Action Mailer allows for you to specify a `before_action`, `after_action` and `around_action`.

- Filters can be specified with a block or a symbol to a method in the mailer class similar to controllers.

- You could use a `before_action` to populate the mail object with defaults, delivery_method_options or insert default headers and attachments.
- You could use an `after_action` to do similar setup as a `before_action` but using instance variables set in your mailer action.

```
class UserMailer < ApplicationMailer
  after_action :set_delivery_options,
               :prevent_delivery_to_guests,
               :set_business_headers

  def feedback_message(business, user)
    @business = business
    @user = user
    mail
  end

  def campaign_message(business, user)
    @business = business
    @user = user
  end

  private

    def set_delivery_options
      # You have access to the mail instance,
      # @business and @user instance variables here
      if @business && @business.has_smtp_settings?
        mail.delivery_method.settings.merge!(@business.smtp_settings)
      end
    end

    def prevent_delivery_to_guests
      if @user && @user.guest?
        mail.perform_deliveries = false
      end
    end

    def set_business_headers
      if @business
        headers["X-SMTPAPI-CATEGORY"] = @business.code
      end
    end
end
```

- Mailer Filters abort further processing if body is set to a non-nil value.

5 Using Action Mailer Helpers